

```

/*
 * Forest Ray
 * May 27, 2025
 * RAGU Lab Internship
 *
 * Using the BNO055
 * Note: instead of the 'master' and 'slave' terminology,
 * I will be using 'central' and 'peripheral'.
 * Stand by for nervous system comparisons.
 */

// include Wire library for I2C communication
// I2C is a commonly used protocol for sending
// and receiving data from various device/sensors
#include <Wire.h>

// define variables
#define BNOAddress 0x28 // we found this using the I2C scanner
// 0x28 is typically the address of the BNO055 according to the adafruit docs
#define OPR_MODE 0x3D // this is the register for the operating mode
// we use Wire.write(OPR_MODE) when we want to change the operating mode
#define UNIT_SEL 0x3B

void setup() {

  Serial.begin(115200);
  // wait for a moment
  delay(1000);

  Serial.println("Beginning Communication...");
  // Wire.begin() initializes the wire library and joins the
  // I2C bus as a central or a peripheral
  // This function should be called only once
  Wire.begin(); // since we input no argument, we join as a central device
  // to join as a peripheral device, use the following
  // Wire.begin(address)
  // where address is the 7-bit peripheral address

  // I2C scanner
  // Here, we are finding the address of the BNO
  for (int add=1; add<=127; add++) {
    // Wire.beginTransmission(address) begins a transmission
    // to the I2C peripheral device with the given address
    Wire.beginTransmission(add);
    // You can queue bytes for transmission with the write() function
    // You can transmit bytes by calling endTransmission

    // Wire.endTransmission() ends a transmission to a peripheral
    // device that was begun by beginTransmission() and
    // transmits the bytes that were queued by write()
  }
}

```

```

// Passing true to endTransmission will send a stop message after
// transmission, releasing the I2C bus.
// Passing false will send a restart message after transmission
// and the bus will not be released.
int reply = Wire.endTransmission();
// Returns:
// 0 - success
// 1 - data too long to fit in transmit buffer
// 2 - received NACK on transmit of address
// 3 - recieved NACK on transmit of data
// 4 - other error
// 5 - timeout

if (reply == 0){ // if endTransmission() returns success
    // print the address of the peripheral device
    Serial.print("BNO address: 0x");
    Serial.println(add, HEX);

    /* Change Operating Mode */
    // Let's put the BNO into inertial measurement unit (IMU) mode
    // first, we have to begin a transmission to the BNO
    Wire.beginTransaction(BNOAddress);
    // the operating mode register is 0x3D, so we write this
    // in order to change the operating mode
    Wire.write(OPR_MODE);
    // to switch to IMU we write xxxx1000b
    Wire.write(1<<3);
    Wire.endTransmission();

    /* Change Units */
    Wire.beginTransaction(BNOAddress);
    Wire.write(UNIT_SEL);
    // switch to degrees xxxxx0xxb
    Wire.write(0<<2);
    Wire.endTransmission();
}

} // end for

} // end void setup

void loop() {

    /* Get Data from the BNO */
    Wire.beginTransaction(BNOAddress);
    // 0x1A is the register for EUL_DATA_X_LSB
    Wire.write(0x1A);
    Wire.endTransmission();
    // we find EUL_DATA_X_LSB at 0x1B
    // 0x1B is the next register so we request 2 bytes

```

```
Wire.requestFrom(BNOAddress, 2);

// read each byte
uint8_t LSB = Wire.read();
uint8_t MSB = Wire.read();

// combine the data and divide by 16 to convert to degrees
// 1 degree = 16 LSB
Serial.println(((int)MSB<<8 | LSB) >> 4);
delay(100);

}
```